```
import re
from collections import Counter
import math



class OptimalBST:
    def __init__(self):
        self.keys = []  # English words
        self.translations = {}  # English to Bengali
        self.p = []  # Successful search probabilities
        self.q = []  # Unsuccessful search probabilities (dummy keys)
        self.root_table = None
        self.e_table = None

    def load_dictionary(self, filename):
        try:
            with open(filename, 'r', encoding='utf-8') as f:
                for line in f:
                    if ',' in line:
                        eng, ben = line.strip().split(',', 1)
                        self.keys.append(eng)
                        self.translations[eng] = ben

            self.keys.sort()
            print(f"Loaded {len(self.keys)} words from dictionary")

        except FileNotFoundError:
            print(f"Dictionary file {filename} not found")

    def analyze_text(self, filename):
        word_count = Counter()
        total_words = 0

        try:
            with open(filename, 'r', encoding='utf-8') as f:
                text = f.read().lower()
                words = re.findall(r'\b[a-z]+\b', text)
                total_words = len(words)
                word_count.update(words)
```

```
                word_count.update(words)

                self.p = [0.0] * len(self.keys)
                for i, word in enumerate(self.keys):
                    count = word_count.get(word, 0)
                    self.p[i] = count / total_words

                self.q = [0.0] * (len(self.keys) + 1)
                total_dummy = 0

                for word, count in word_count.items():
                    if word < self.keys[0]:
                        total_dummy += count
                self.q[0] = total_dummy / total_words

                for i in range(1, len(self.keys)):
                    total_dummy = 0
                    for word, count in word_count.items():
                        if self.keys[i-1] < word < self.keys[i]:
                            total_dummy += count
                    self.q[i] = total_dummy / total_words

                total_dummy = 0
                for word, count in word_count.items():
                    if word > self.keys[-1]:
                        total_dummy += count
                self.q[len(self.keys)] = total_dummy / total_words

                print(f"Analyzed {total_words} words")
                print(f"Successful probabilities sum: {sum(self.p):.3f}")
                print(f"Unsuccessful probabilities sum: {sum(self.q):.3f}")

            except FileNotFoundError:
                print(f"Input file {filename} not found")

        def build_optimal_bst(self):
            n = len(self.keys)
            if n == 0:
                return

            self.e_table = [[0.0] * (n + 1) for _ in range(n + 2)]
```

```
        self.w_table = [[0.0] * (n + 1) for _ in range(n + 2)]
        self.root_table = [[0] * (n + 1) for _ in range(n + 1)]

        for i in range(1, n + 2):
            self.e_table[i][i - 1] = self.q[i - 1]
            self.w_table[i][i - 1] = self.q[i - 1]

        for length in range(1, n + 1):
            for i in range(1, n - length + 2):
                j = i + length - 1
                self.e_table[i][j] = float('inf')
                self.w_table[i][j] = self.w_table[i][j - 1] + self.p[j - 1] + self.q[j]

                for r in range(i, j + 1):
                    cost = self.e_table[i][r - 1] + self.e_table[r + 1][j] + self.w_table[i][j]
                    if cost < self.e_table[i][j]:
                        self.e_table[i][j] = cost
                        self.root_table[i][j] = r

        print(f"Optimal BST expected cost: {self.e_table[1][n]:.6f}")

    def construct_tree(self, i, j):
        if i > j:
            return None

        root_index = self.root_table[i][j] - 1
        root_word = self.keys[root_index]

        return {
            'word': root_word,
            'translation': self.translations[root_word],
            'probability': self.p[root_index],
            'left': self.construct_tree(i, self.root_table[i][j] - 1),
            'right': self.construct_tree(self.root_table[i][j] + 1, j)
        }

    def print_tree(self, node, level=0, prefix="Root:"):
        if node is not None:
            print(" " * (level * 4) + prefix + f" {node['word']} "
                  f"(p: {node['probability']:.4f})")
            self.print_tree(node['left'], level + 1, "L:")
```

```
            self.print_tree(node['right'], level + 1, "R:")

    def search_cost(self, word, node=None, current_depth=1):
        if node is None:
            # Build tree if not already built
            root_node = self.construct_tree(1, len(self.keys))
            return self.search_cost(word, root_node, 1)

        if word == node['word']:
            return current_depth

        if word < node['word']:
            if node['left'] is None:
                return current_depth + 1  # Would go to dummy node
            return self.search_cost(word, node['left'], current_depth + 1)
        else:
            if node['right'] is None:
                return current_depth + 1  # Would go to dummy node
            return self.search_cost(word, node['right'], current_depth + 1)

class AVLTree:
    class Node:
        def __init__(self, word, translation, prob):
            self.word = word
            self.translation = translation
            self.prob = prob
            self.left = None
            self.right = None
            self.height = 1

    def __init__(self):
        self.root = None

    def height(self, node):
        return node.height if node else 0

    def update_height(self, node):
        node.height = 1 + max(self.height(node.left), self.height(node.right))

    def balance_factor(self, node):
        return self.height(node.left) - self.height(node.right) if node else 0
```

```
        return self.height(node.left) - self.height(node.right) if node else 0

    def rotate_right(self, y):
        x = y.left
        T2 = x.right

        x.right = y
        y.left = T2

        self.update_height(y)
        self.update_height(x)

        return x

    def rotate_left(self, x):
        y = x.right
        T2 = y.left

        y.left = x
        x.right = T2

        self.update_height(x)
        self.update_height(y)

        return y

    def insert(self, node, word, translation, prob):
        if node is None:
            return self.Node(word, translation, prob)

        if word < node.word:
            node.left = self.insert(node.left, word, translation, prob)
        else:
            node.right = self.insert(node.right, word, translation, prob)

        self.update_height(node)
        balance = self.balance_factor(node)

        if balance > 1 and word < node.left.word:
            return self.rotate_right(node)

        if balance < -1 and word > node.right.word:
```

```
            if balance < -1 and word > node.right.word:
                return self.rotate_left(node)

            if balance > 1 and word > node.left.word:
                node.left = self.rotate_left(node.left)
                return self.rotate_right(node)

            if balance < -1 and word < node.right.word:
                node.right = self.rotate_right(node.right)
                return self.rotate_left(node)

            return node

        def build_avl(self, words, translations, probabilities):
            import random
            shuffled_words = words.copy()
            random.shuffle(shuffled_words)

            for word in shuffled_words:
                self.root = self.insert(self.root, word, translations[word], probabilities[words.index(word)])

        def search_cost(self, word, node=None, current_depth=1):
            if node is None:
                node = self.root

            if node is None:
                return current_depth

            if word == node.word:
                return current_depth

            if word < node.word:
                return self.search_cost(word, node.left, current_depth + 1)
            else:
                return self.search_cost(word, node.right, current_depth + 1)

    def main():
        print("=== Optimal BST vs AVL Tree Comparison ===\n")

        optimal_bst = OptimalBST()
        optimal_bst.load_dictionary('/content/dictionary.txt')
```

```
optimal_bst.analyze_text('/content/input.txt')
optimal_bst.build_optimal_bst()

avl_tree = AVLTree()
avl_tree.build_avl(optimal_bst.keys, optimal_bst.translations, optimal_bst.p)

print("\n=== Search Cost Comparison ===")

word_prob_pairs = [(optimal_bst.keys[i], optimal_bst.p[i]) for i in range(len(optimal_bst.keys))]
word_prob_pairs.sort(key=lambda x: x[1], reverse=True)

test_words = [word for word, prob in word_prob_pairs[:10]]

total_optimal_cost = 0
total_avl_cost = 0

print(f"{'Word':<15} {'Prob':<8} {'Optimal':<8} {'AVL':<8} {'Difference':<10}")
print("-" * 55)

for word in test_words:
    optimal_cost = optimal_bst.search_cost(word)
    avl_cost = avl_tree.search_cost(word)
    total_optimal_cost += optimal_cost
    total_avl_cost += avl_cost

    prob = optimal_bst.p[optimal_bst.keys.index(word)]
    diff = avl_cost - optimal_cost

    print(f"{word:<15} {prob:<8.4f} {optimal_cost:<8} {avl_cost:<8} {diff:>+8}")

print("-" * 55)
print(f"{'TOTAL':<15} {'':<8} {total_optimal_cost:<8} {total_avl_cost:<8} {total_avl_cost - total_optimal_cost:>+8}

print(f"\n=== Final Results ===")
print(f"Optimal BST total cost: {total_optimal_cost}")
print(f"AVL Tree total cost: {total_avl_cost}")

if total_optimal_cost < total_avl_cost:
    efficiency = total_avl_cost / total_optimal_cost
    print(f"Optimal BST is {efficiency:.2f}x more efficient!")
else:
```

```
        print("Unexpected result: AVL tree performed better")
        print("This might happen if:")
        print("- Input text has uniform word distribution")
        print("- Dictionary words are not frequently used")
        print("- Random AVL construction created a very balanced tree")

if __name__ == "__main__":
    main()
```

```
=== Optimal BST vs AVL Tree Comparison ===

Loaded 345 words from dictionary
Analyzed 61 words
Successful probabilities sum: 0.344
Unsuccessful probabilities sum: 0.721
Optimal BST expected cost: 5.967213

=== Search Cost Comparison ===
Word            Prob     Optimal  AVL      Difference
-----------------------------------------------------
book            0.0164   4        7              +3
book            0.0164   4        7              +3
book            0.0164   4        7              +3
boy             0.0164   6        7              +1
cat             0.0164   5        7              +2
dog             0.0164   2        7              +5
elephant        0.0164   6        7              +1
garden          0.0164   5        7              +2
house           0.0164   4        8              +4
house           0.0164   4        8              +4
-----------------------------------------------------
TOTAL                    44       72            +28

=== Final Results ===
Optimal BST total cost: 44
AVL Tree total cost: 72
Optimal BST is 1.64x more efficient!
```